

ui.time_input 全面详解

`ui.time_input` 是 NiceGUI 3.3.0 版本新增的时间输入组件，基于 Quasar 的 `QInput` 组件扩展并集成时间选择器，支持单时间点选择、文本输入与选择器交互同步，提供标签自定义、占位提示、值变更回调等核心功能，适用于表单时间录入场景（如预约时间设置、任务开始时间、日程安排等）。

一、核心初始化参数

初始化 `ui.time_input` 仅需 4 个核心参数，简洁覆盖基础时间输入需求，参数设计兼顾实用性与易用性：

参数名	说明
<code>label</code>	组件显示标签，用于提示用户输入用途（如 "预约时间"、"截止时间"）
<code>placeholder</code>	未选择时间时显示的占位文本（如 "请选择时间"、"HH:MM"）
<code>value</code>	初始时间值，格式为 <code>HH:MM</code> 字符串（如 <code>'12:30'</code> 、 <code>'08:45'</code> ），支持 24 小时制
<code>on_change</code>	时间值变更时触发的回调函数，事件对象通过 <code>e.value</code> 获取当前选中时间

基础使用示例

```
from nicegui import ui

# 初始化时间选择器，初始值为 12:30
time_picker = ui.time_input(
    label='会议时间',
    placeholder='请选择会议开始时间',
    value='12:30',
    on_change=lambda e: ui.notify(f'会议时间已设置为: {e.value}')
)

# 实时显示当前选中时间（通过数据绑定）
ui.label().bind_text_from(time_picker, 'value', lambda v: f'当前选择: {v}')

ui.run()
```

二、组件核心属性

`ui.time_input` 继承 NiceGUI 基础元素的核心属性，支持动态状态控制、样式自定义、DOM 标识等能力，关键属性如下：

属性名	类型	说明
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观（如宽度、边框、间距）
<code>client</code>	<code>Client</code>	组件所属的客户端实例（多客户端场景下的隔离标识）

属性名	类型	说明
<code>enabled</code>	<code>BindableProperty</code>	组件是否启用（可绑定， <code>False</code> 时禁止用户交互，支持动态启用 / 禁用）
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增，用于精准 DOM 操作）
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除（只读属性，用于判断组件生命周期状态）
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件（只读属性，用于调试或事件控制场景）
<code>label</code>	<code>BindableProperty</code>	组件标签（支持动态绑定，可通过 <code>set_label</code> 方法修改）
<code>parent_slot</code>	<code>Slot None</code>	组件的父插槽（可设置，用于复杂布局中的插槽嵌套）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性（用于扩展输入框行为，如 <code>dense</code> 紧凑模式、 <code>readonly</code> 只读模式）
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式（如 <code>width: 200px; margin: 10px 0</code> ）
<code>value</code>	<code>BindableProperty</code>	当前选中的时间值（ <code>HH:MM</code> 格式字符串，支持动态绑定，可通过 <code>set_value</code> 方法修改）
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（支持动态绑定， <code>False</code> 时隐藏组件）

属性操作示例

```
from nicegui import ui

# 初始化时间选择器
time_picker = ui.time_input(label='初始标签', value='12:30')

# 按钮控制组件状态
ui.button('禁用选择器', on_click=time_picker.disable)
ui.button('启用选择器', on_click=time_picker.enable)
ui.button('修改标签', on_click=lambda: time_picker.set_label('更新后的时间标签'))
ui.button('设置为 09:00', on_click=lambda: time_picker.set_value('09:00'))
ui.button('隐藏选择器', on_click=lambda: time_picker.set_visibility(False))

ui.run()
```

三、核心方法

`ui.time_input` 提供完善的方法用于组件状态控制、数据绑定、样式自定义等，按功能分类如下：

1. 状态控制方法

用于直接修改组件的启用状态、可见性、标签、值等基础属性：

方法名	说明
<code>enable()</code>	启用组件（允许用户交互，输入框可编辑、选择器可打开）
<code>disable()</code>	禁用组件（禁止用户交互，输入框变灰，无法打开选择器）
<code>set_enabled(value: bool)</code>	动态设置启用状态（ <code>True</code> 启用， <code>False</code> 禁用）
<code>set_label(label: str None)</code>	修改组件标签（传 <code>None</code> 隐藏标签）
<code>set_value(value: str)</code>	设置时间值（需传入 <code>HH:MM</code> 格式字符串，如 <code>'14:20'</code> ）
<code>set_visibility(visible: bool)</code>	动态设置组件可见性（ <code>True</code> 显示， <code>False</code> 隐藏）
<code>clear()</code>	清除所有子元素（极少用于时间输入框，适用于嵌套复杂内容场景）
<code>delete()</code>	删除组件及其所有子元素（释放资源，生命周期结束）

2. 数据绑定方法

支持将组件的 `enabled`、`label`、`value`、`visible` 等属性与目标对象属性进行单向 / 双向绑定，实现数据同步：

方法名	说明
<code>bind_enabled(...)</code>	双向绑定组件启用状态与目标对象属性
<code>bind_enabled_from(...)</code>	单向绑定（目标对象 → 组件）启用状态
<code>bind_enabled_to(...)</code>	单向绑定（组件 → 目标对象）启用状态
<code>bind_label(...)</code>	双向绑定组件标签与目标对象属性
<code>bind_label_from(...)</code>	单向绑定（目标对象 → 组件）标签
<code>bind_label_to(...)</code>	单向绑定（组件 → 目标对象）标签
<code>bind_value(...)</code>	双向绑定组件时间值与目标对象属性（核心绑定方法）
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）时间值
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）时间值
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性

绑定示例（与数据类同步）

```
from dataclasses import dataclass
from nicegui import ui

# 定义数据类（存储业务数据）
@dataclass
class Schedule:
    start_time: str = '10:00' # 与时间选择器绑定的属性

schedule = Schedule()

# 时间选择器与 schedule.start_time 双向绑定
time_picker = ui.time_input(label='日程开始时间').bind_value(schedule, 'start_time')

# 显示绑定的当前值
ui.label().bind_text_from(schedule, 'start_time', lambda v: f'日程开始时间: {v}')

# 按钮修改数据类属性（间接同步到时间选择器）
ui.button('推迟1小时', on_click=lambda: setattr(schedule, 'start_time', '11:00'))

ui.run()
```

3. 样式与扩展方法

用于自定义组件外观、添加资源或辅助功能：

方法名	说明
<code>classes(add/remove/toggle/replace)</code>	新增 / 移除 / 切换 / 替换组件的 HTML 类（如 <code>time_picker.classes('w-40 border-blue-500')</code> ）
<code>style(add/remove/replace)</code>	新增 / 移除 / 替换组件的内联 CSS 样式（如 <code>time_picker.style('font-size: 14px;')</code> ）
<code>default_classes(...)</code>	全局修改该类组件的默认 HTML 类（需在实例化前调用，如统一设置宽度）
<code>default_style(...)</code>	全局修改该类组件的默认 CSS 样式（需在实例化前调用）
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示（鼠标悬浮时显示，如 <code>tooltip('选择会议开始时间')</code> ）
<code>add_resource(path)</code>	为组件添加资源文件（如自定义 CSS/JS，用于扩展样式或功能）
<code>mark(*markers)</code>	为组件添加标记（用于测试查询或依赖管理）

样式自定义示例

```
# 全局设置所有时间输入框的默认样式（实例化前调用）
ui.time_input.default_classes(add='shadow-sm border-gray-300 rounded-lg')
ui.time_input.default_style(add='margin: 8px 0; padding: 4px;')

# 实例化时添加局部样式
time_picker = ui.time_input(
    label='样式自定义示例',
    value='15:45',
    placeholder='选择时间'
)
time_picker.classes('w-48 border-green-500') # 局部类（宽度、边框颜色）
time_picker.style('font-size: 15px;') # 局部内联样式（字体大小）
time_picker.tooltip('支持直接输入 HH:MM 格式或点击选择') # 添加提示

ui.run()
```

4. 其他实用方法

方法名	说明
<code>ancestors(include_self)</code>	迭代组件的祖先元素（ <code>include_self=True</code> 包含自身）
<code>descendants(include_self)</code>	迭代组件的子元素（ <code>include_self=True</code> 包含自身）
<code>get_computed_prop(prop_name, timeout)</code>	获取计算属性（需异步等待，如获取组件实际渲染后的宽度）
<code>move(target_container, target_index, target_slot)</code>	移动组件到其他容器（用于动态布局调整）
<code>on(type, handler, ...)</code>	订阅通用 DOM 事件（如点击、鼠标悬浮、聚焦等）
<code>on_value_change(callback)</code>	绑定时间值变更事件（与初始化 <code>on_change</code> 参数等价）
<code>remove(element)</code>	移除子元素（极少使用）
<code>run_method(name, *args)</code>	运行客户端方法（如调用底层 Quasar 组件的原生方法）
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据，如动态修改样式后刷新）

四、事件处理

1. 核心事件：时间值变更（on_change/on_value_change）

时间值变更事件是 `ui.time_input` 的核心事件，支持两种绑定方式（功能等价）：

```
# 方式1: 初始化时通过 on_change 参数绑定
ui.time_input(
    label='事件绑定示例',
    on_change=lambda e: print(f'时间变更为: {e.value}')
)

# 方式2: 通过 on_value_change 方法绑定
time_picker = ui.time_input(label='事件绑定示例')
time_picker.on_value_change(lambda e: print(f'时间变更为: {e.value}'))
```

2. 通用事件订阅（on 方法）

通过 `on()` 方法订阅其他 DOM 事件（如点击、鼠标悬浮、聚焦、失焦等），支持客户端 JS 处理或服务端 Python 处理：

```
from nicegui import ui

time_picker = ui.time_input(label='通用事件示例', value='09:30')

# 订阅聚焦事件（服务端处理）
time_picker.on('focus', lambda: print('时间输入框获得焦点'))

# 订阅失焦事件（客户端 JS 处理，验证时间格式）
time_picker.on(
    'blur',
    js_handler='(e) => { const val = e.target.value; const regex = /^[01]?[0-9]|2[0-3]):[0-5][0-9]$/; if (!regex.test(val) && val) alert("时间格式错误，请输入 HH:MM 格式"); }'
)

ui.run()
```

五、版本兼容性说明

特性 / 属性	支持版本
<code>ui.time_input</code> 组件本身	3.3.0+（新增）
<code>html_id</code> 属性	2.16.0+
<code>toggle</code> 参数（ <code>default_classes</code> ）	2.7.0+
<code>strict</code> 参数（绑定方法）	3.0.0+
同时指定 Python 和 JS 事件处理（ <code>on</code> 方法）	2.18.0+

六、关键注意事项

1. 时间格式规范：

初始值、设置值需严格遵循 `HH:MM` 格式（如 `'08:00'` 而非 `'8:0'`），支持 24 小时制（`00:00` 表示午夜，`23:59` 表示深夜）；直接输入时组件会自动校验格式，不符合规范则无法确认。

2. 交互同步特性：

支持两种输入方式并实时同步：

- 点击组件打开时间选择器，可视化选择小时、分钟；
- 直接在输入框输入 `HH:MM` 格式文本，输入完成后自动同步到组件值。

3. Quasar 原生属性扩展：

通过 `props` 方法可扩展 Quasar `QInput` 的原生属性，例如：

```
# 启用紧凑模式、只读模式（仅允许选择器选择，禁止手动输入）
ui.time_input(label='只读模式示例', value='10:15').props('dense readonly')
```

总结

`ui.time_input` 是 NiceGUI 3.3.0+ 版本推出的高效时间输入组件，核心优势在于：

- 一体化设计，集成输入框与时间选择器，兼顾手动输入灵活性与可视化选择便捷性；
- 支持丰富的样式自定义与数据绑定，可无缝融入业务数据模型；
- 基于 Quasar 组件，交互流畅、跨浏览器兼容，且格式校验自动生效；
- 初始化参数简洁，开箱即用，无需复杂配置即可满足大多数时间输入场景。

适用于表单录入、日程安排、预约系统等需要精准时间输入的场景，结合 NiceGUI 的按钮、标签、卡片等组件可快速构建直观的时间交互界面。